Unit-2

Programming The Basic Computer

Introduction

- The 25 instructions of the basic computer are shown in Table.
- The first seven instructions are memory-reference instructions and the other 18 are register-reference and input-output instructions.
- A memory- reference instruction has three parts: a mode bit, an operation code of three bits, and a 12-bit address. The first hexadecimal digit of a memory-reference instruction includes the mode bit and the operation code. The other three digits specify the address. In an indirect address instruction the mode bit is 1 and the first hexadecimal digit ranges in value from 8 to E. In a direct mode, the range is from 0 to 6.
- The other 18 instructions have a 16-bit operation code. The code for each instruction is listed as a four-digit hexadecimal number. The first digit of a register-reference instruction is always 7.
- The first digit of an input-output instruction is always F.

Computer Instruction

Symbol	Hexadecimal code	Description
AND	0 or 8	AND M to AC
ADD	1 or 9	Add M to AC, carry to E
LDA	2 or A	Load AC from M
STA	3 or B	Store AC in M
BUN	4 or C	Branch unconditionally to m
BSA	5 or D	Save return address in m and branch to $m + 1$
ISZ	6 or E	Increment M and skip if zero
CLA	7800	Clear AC
CLE	7400	Clear E
CMA	7200	Complement AC
CME	7100	Complement E
CIR	7080	Circulate right E and AC
CIL	7040	Circulate left E and AC
INC	7020	Increment AC,
SPA	7010	Skip if AC is positive
SNA	7008	Skip if AC is negative
SZA	7004	Skip if AC is zero
SZE	7002	Skip if E is zero
HLT	7001	Halt computer
INP	F800	Input information and clear flag
OUT	F400	Output information and clear flag
SKI	F200	Skip if input flag is on
SKO	F100	Skip if output flag is on
ION	F080	Turn interrupt on
IOF	F040	Turn interrupt off

Machine Language

A program is a list of instructions or statements for directing the computer to perform a required data-processing task. There are various types of programming languages that one may write for a computer, but the computer can execute programs only when they are represented internally in binary form.

Program written in any other language must be translated to the binary representation of instructions before they can be executed by the computer. Programs written for a computer may be in one of the following categories:

1. Binary code. This is a sequence of instructions and operands in binary that list the exact representation of instructions as they appear in computer memory.

2. Octal or hexadecimal code. This is an equivalent translation of the binary code to octal or hexadecimal representation.

3. Symbolic code. The user employs symbols (letters, numerals, or special characters) for the operation part, the address part, and other parts of the instruction code. Each symbolic instruction can be translated into one binary coded instruction. This translation is done by a special program called an **assembler**. Because an assembler translates the symbols, this type of symbolic program is referred to as an **assembly language program**.

4. High-level programming languages. These are special languages developed to reflect the procedures used in the solution of a problem rather than be concerned with the computer hardware behavior. An example of a high-level programming language is Fortran. The program is written in a sequence of statements in a form that people prefer to think in when solving a problem. However, each statement must be translated into a sequence of binary instructions before the program can be executed in a computer. **The program that translates a high-level language program to binary is called a compiler.**

Binary Program to Add Two Numbers

Location	Instruction code
0	0010 0000 0000 0100
1	0001 0000 0000 0101
10	0011 0000 0000 0110
11	0111 0000 0000 0001
100	0000 0000 0101 0011
101	1111 1111 1110 1001
110	0000 0000 0000 0000

Hexadecimal Program to Add Two Numbers

Location	Instruction
000	2004
001	1005
002	3006
003	7001
004	0053
005	FFE9
006	0000

Program with Symbolic Operation codes

Location	Instruction	Comments
000	LDA 004	Load first operand into *AC*
001	ADD 005	Add second operand to *AC*
002	STA 006	Store sum in location 006
003	HLT	Halt computer
004	0053	First operand
005	FFE9	Second operand (negative)
006	0000	Store sum here

Assembly language Program to Add Two Numbers

	ORG 0	/Origin of program is location 0
	LDA A	/Load operand from location *A*
	ADD B	/Add operand from location *B*
	STA C	/Store sum in location *C*
	HLT	/Halt computer
A,	DEC 83	/Decimal operand
B,	DEC −23	/Decimal operand
C,	DEC 0	/Sum stored in location *C*
	END	/End of symbolic program

Replace each hexadecimal address by a symbolic address and each hexadecimal operand by a decimal operand. This is convenient because one usually does not know exactly the numeric memory location of operands while writing a program. If the operands are placed in memory following the instructions, and if the length of the program is not known in advance, the numerical location of operands is not known until the end of the program is reached. In addition, decimal numbers are more familiar than their hexadecimal equivalents.

The program in Table is the assembly-language program for adding two numbers. The symbol ORG specify an origin, that is, the memory location of the next instruction below it. The next three lines have symbolic addresses. Their value is specified by their being present as a label in the first column. Decimal operands are specified following the symbol DEC. The last line has the symbol END indicating the end of the program.

ASSEMBLY LANGUAGE

Assembly language is a low level programming language and programming language is defined by a set of rules. In this, Programs written in assembly languages are compiled by an assembler. Every assembler has its own assembly language, which is designed for one specific computer architecture.

The basic unit of assembly language program is a line of code. Assembly Language mainly consists of mnemonic processor instructions or data, and other statements or instructions.

Rules of the Language

Each line of an assembly language program is arranged in three columns called fields. The fields specify the following information.

1. The **label field** may be empty or it may specify a symbolic address.
2. The **instruction field** specifies a machine instruction or a pseudo-instruction.
3. The **comment field** may be empty or it may include a comment.

A **symbolic address** consists of one, two, or three, but not more than three alphanumeric characters. The first character must be a letter; the next two may be letters or numerals. A symbolic address in the label field is terminated by a comma so that it will be recognized as a label by the assembler.

The **instruction field** in an assembly language program may specify one of the following items:

1. A memory-reference instruction (MRI)
2. A register-reference or input-output instruction (non-MRI)
3. A pseudo-instruction with or without an operand

A pseudoinstruction is not a machine instruction but rather an instruction to the assembler giving information about some phase of the translation.

- The ORG (origin) pseudoinstruction informs the assembler that the instruction or operand in the following line is to be placed in a memory location specified by the number next to ORG.
- The END symbol is placed at the end of the program to inform the assembler that the program is terminated.
- The other two pseudoinstructions specify the radix of the operand and tell the assembler how to convert the listed number to a binary number.

The third field in a program is reserved for comments. A line of code may or may not have a comment, but if it has, it must be preceded by a slash for the assembler to recognize the beginning of a comment field. Comments are useful for explaining the program and are helpful in understanding the step- by-step procedure taken by the program. Comments are inserted for explanation purposes only and are neglected during the binary translation process

TABLE 6-7 Definition of Pseudoinstructions

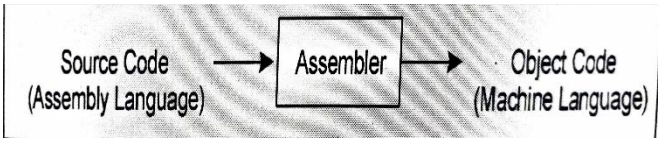
Symbol	Information for the Assembler
ORG N	Hexadecimal number N is the memory location for the instruction or operand listed in the following line
END	Denotes the end of symbolic program
DEC N	Signed decimal number N to be converted to binary
HEX N	Hexadecimal number N to be converted to binary

Example: Assembly language Program to Subtract Two Numbers

```
        ORG 100     /Origin of program is location 100
        LDA SUB     /Load subtrahend to AC
        CMA         /Complement AC
        INC         /Increment AC
        ADD MIN     /Add minuend to AC
        STA DIF     /Store difference
        HLT         /Halt computer
MIN,    DEC 83      /Minuend
SUB,    DEC -23     /Subtrahend
DIF,    HEX 0       /Difference stored here
        END         /End of symbolic program
```

Assembler:

An assembler is a program that accepts a symbolic language program and produces its binary machine language equivalent. The input symbolic program is called the source program and the resulting binary program is called the object program. The assembler is a program that operates on character strings and produces an equivalent binary interpretation.



Hexadecimal Character code

Character	Code	Character	Code	Character	Code	
A	41	Q	51	6	36	
B	42	R	52	7	37	
C	43	S	53	8	38	
D	44	T	54	9	39	
E	45	U	55	space	20	
F	46	V	56	(	28	
G	47	W	57	)	29	
H	48	X	58	*	2A	
I	49	Y	59	+	2B	
J	4A	Z	5A	,	2C	
K	4B	0	30	−	2D	
L	4C	1	31	.	2E	
M	4D	2	32	/	2F	
N	4E	3	33	=	3D	
O	4F	4	34	CR	0D	(carriage
P	50	5	35			return)

Computer Representation of the Line of Code: PL3, LDA SUB I

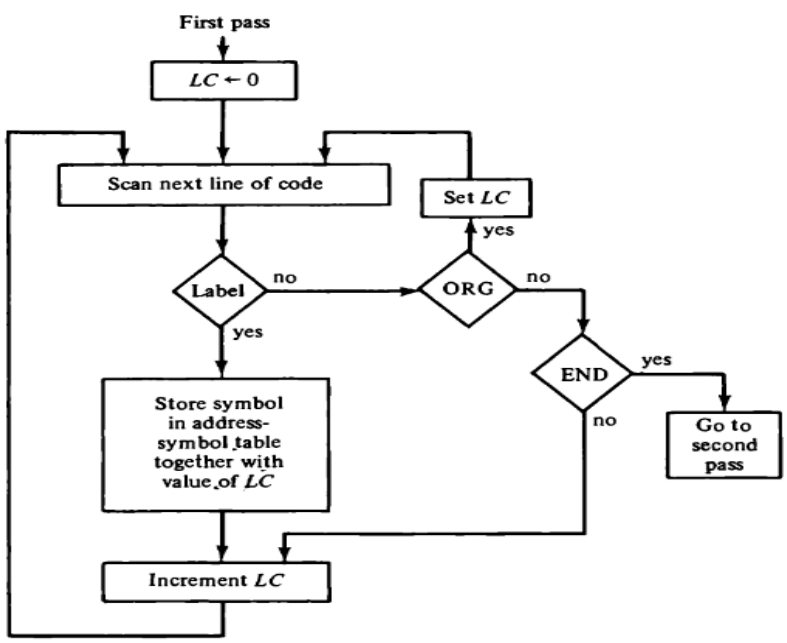
Memory word	Symbol	Hexadecimal code	Binary representation
1	P L	50 4C	0101 0000 0100 1100
2	3 ,	33 2C	0011 0011 0010 1100
3	L D	4C 44	0100 1100 0100 0100
4	A	41 20	0100 0001 0010 0000
5	S U	53 55	0101 0011 0101 0101
6	B	42 20	0100 0010 0010 0000
7	I CR	49 0D	0100 1001 0000 1101

There are two types of Assembler:

- First pass assembler
- Second pass assembler

First pass assembler

A two-pass assembler scans the entire symbolic program twice. **During the first pass, it generates a table that correlates all user-defined address symbols with their binary equivalent value**. The binary translation is done during the second pass. To keep track of the location of instructions, the assembler uses a memory word called a **location counter** (LC). The content of LC stores the value of the memory location assigned to the instruction or operand presently being processed. The ORG pseudoinstruction initializes the location counter to the value of the first location. Since instructions are stored in sequential locations, the content of LC is incremented by 1 after processing each line of code.



The tasks performed by the assembler during the first pass are described in the flowchart. LC is initially set to 0. A line of symbolic code is analyzed to determine if it has a label (by the presence of a comma). If the line of code has no label, the assembler checks the symbol in the instruction field. If it contains an ORG pseudoinstruction, the assembler sets LC to the number that follows ORG and goes back to process the next line. If the line has an END pseudoinstruction, the assembler terminates the first pass and goes to the second pass. (Note that a line with ORG or END should not have a label.) If the line of code contains a label, it is stored in the address symbol table together with its binary equivalent number specified by the content of LC. Nothing is stored in the table if no label is encountered. LC is then incremented by 1 and a new line of code is processed.

The assembler generates the address symbol table listed in Table. Each label symbol is stored in two memory locations and is terminated by a comma. If the label contains less than three characters, the memory locations are filled with the code for space. The value found in LC while the line was processed is stored in the next sequential memory location. The program has three symbolic addresses: MIN, SUB, and DIF. These symbols represent 12-bit addresses equivalent to hexadecimal 106, 107, and 108,

Address Symbol Table

Memory word	Symbol or (LC)*	Hexadecimal code	Binary representation
1	M I	4D 49	0100 1101 0100 1001
2	N ,	4E 2C	0100 1110 0010 1100
3	(LC)	01 06	0000 0001 0000 0110
4	S U	53 55	0101 0011 0101 0101
5	B ,	42 2C	0100 0010 0010 1100
6	(LC)	01 07	0000 0001 0000 0111
7	D I	44 49	0100 0100 0100 1001
8	F ,	46 2C	0100 0110 0010 1100
9	(LC)	01 08	0000 0001 0000 1000

* (LC) designates content of location counter.

Second pass assembler

Machine instructions are translated during the second pass by means of table-lookup procedures. A table-lookup procedure is a search of table entries to determine whether a specific item matches one of the items stored in the table. The assembler uses four tables. Any symbol that is encountered in the program must be available as an entry in one of these tables; otherwise, the symbol cannot be interpreted.

The assembler uses four tables

1. Pseudoinstruction table.
2. MRI table.
3. Non-MRI table.
4. Address symbol table.

Program Loops

A program loop is a sequence of instructions that are executed many times, each time with a different set of data. The following is an example of a Fortran program that forms the sum of 100 integer numbers.

```
DIMENSION A(100)
INTEGER SUM, A
SUM = 0
DO J= 1, 100
SUM = SUM + A(J)
```

Statement number 3 is executed 100 times, each time with a different operand A(J) for J = 1, 2, ..., 100.

A system program that translates a program written in a high-level programming language such as the above to a machine language program is called a **compiler.**

Symbolic Program to add 100 numbers

Line			
1		ORG 100	/Origin of program is HEX 100
2		LDA ADS	/Load first address of operands
3		STA PTR	/Store in pointer
4		LDA NBR	/Load minus 100
5		STA CTR	/Store in counter
6		CLA	/Clear accumulator
7	LOP,	ADD PTR I	/Add an operand to *AC*
8		ISZ PTR	/Increment pointer
9		ISZ CTR	/Increment counter
10		BUN LOP	/Repeat loop again
11		STA SUM	/Store sum
12		HLT	/Halt
13	ADS,	HEX 150	/First address of operands
14	PTR,	HEX 0	/This location reserved for a pointer
15	NBR,	DEC −100	/Constant to initialized counter
16	CTR,	HEX 0	/This location reserved for a counter
17	SUM,	HEX 0	/Sum is stored here
18		ORG 150	/Origin of operands is HEX 150
19		DEC 75	/First operand
•			
•			
•			
118		DEC 23	/Last operand
119		END	/End of symbolic program

The first statement in the Fortran program is a DIMENSION statement. This statement instructs the compiler to reserve 100 words of memory for 100 operands. The second statement informs the compiler that the numbers are integers. The compiler reserves locations $(150)_{16}$ to $(1B3)_{16}$ for the 100 operands. These reserved memory words are listed in lines 19 to 118 in the translated program. This is done by the ORG pseudoinstruction in line 18, which specifies the origin of the operands.

The program loop specified by the DO statement is translated to the sequence of instructions listed in lines 7 through 10. Line 7 specifies an indirect ADD instruction because it has the symbol I. The address of the current operand is stored in location PTR. When this location is addressed indirectly the computer takes

the content of PTR to be the address of the operand. As a result, the operand in location 150 is added to the accumulator. Location PTR is then incremented with the ISZ instruction in line 8, so its value changes to the value of the address of the next sequential operand. Location CTR is incremented in line 9, and if it is not zero, the computer does not skip the next instruction. The next instruction is a branch (BUN) instruction to the beginning of the loop, so the computer returns to repeat the loop once again. When location CTR reaches zero (after the loop is executed 100 times), the next instruction is skipped and the computer executes the instructions in lines 11 and 12. The sum formed in the accumulator is stored in SUM and the computer halts.

Programming Arithmetic and Logic Operations

The number of instructions available in a computer may be a few hundred in a large system or a few dozen in a small one. Some computers perform a given operation with one machine instruction; others may require a large number of machine instructions to perform the same operation. As an illustration, consider the four basic arithmetic operations. Some computers have machine instructions to add, subtract, multiply, and divide. Others, such as the basic computer, have only one arithmetic instruction, such as ADD.

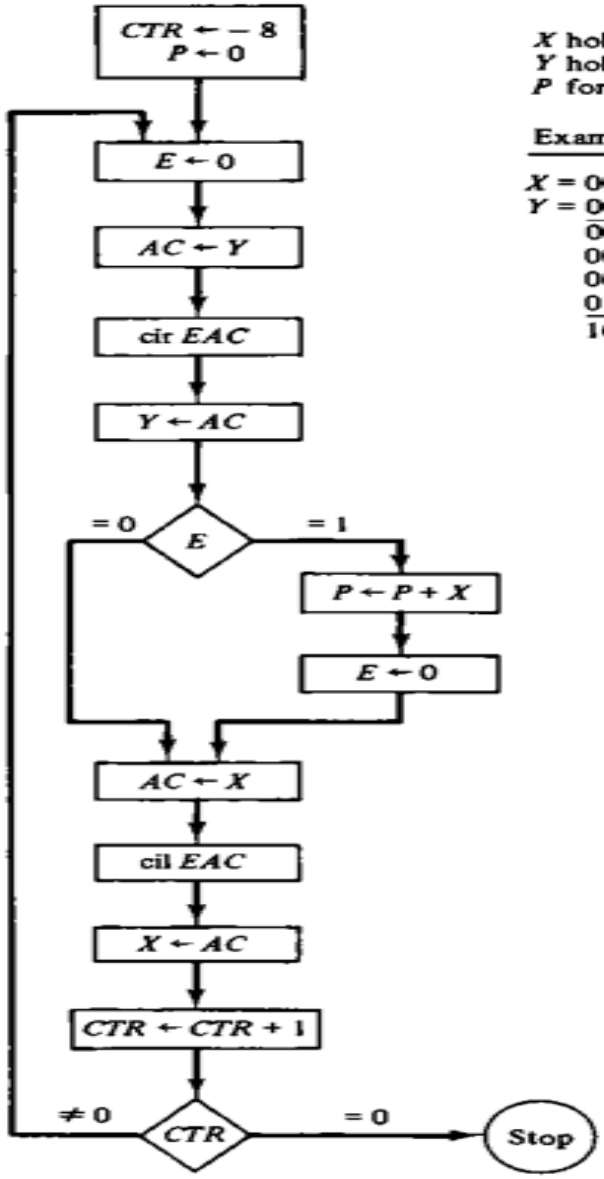
Operations that are implemented in a computer with one machine instruction are said to be implemented by hardware. Operations implemented by a set of instructions that constitute a program are said to be implemented by software.

The multiplication process consists of checking the bits of the multiplier Y and adding the multiplicand X as many times as there are 1's in Y, provided that the value of X is shifted left from one line to the next. Since the computer can add only two numbers at a time, we reserve a memory location, denoted by P, to store intermediate sums. The intermediate sums are called partial products since they hold a partial product until all numbers are added. The partial product P starts with zero. The multiplicand X is added to the content of P for each bit of the multiplier Y that is 1. The value of X is shifted left after checking each bit of the multiplier. The final value in P forms the product.

Flow chart for multiplication operation.

The program has a loop that is traversed eight times, once for each significant bit of the multiplier. Initially, location X holds the multiplicand and location Y holds the multiplier. A counter CTR is set to -8 and location P is cleared to zero.

The multiplier bit can be checked if it is transferred to the E register. This is done by clearing E, loading the value of Y into the AC, circulating right E and AC and storing the shifted number back into location Y. This bit stored in E is the low-order bit of the multiplier. We now check the value of E. If it is 1, the multiplicand X is added to the partial product P. If it is 0, the partial product does not change. We then shift the value of X once to the left by loading it into the AC and circulating left E and AC. The loop is repeated eight times by incrementing location CTR and checking when it reaches zero. When the counter reaches zero, the program exits from the loop with the product stored in location P.



X holds the multiplicand
Y holds the multiplier
P forms the product

Example with four significant digits

		P
X =	0000 1111	0000 0000
Y =	0000 1011	0000 1111
	0000 1111	0010 1101
	0001 1110	0010 1101
	0000 0000	1010 0101
	0111 1000	
	1010 0101	

Logic Operations

The basic computer has three machine instructions that perform logic operations: AND, CMA, and CLA. The LDA instruction may be considered as a logic operation that transfers a logic operand into the AC.

A program that forms the OR operation of two logic operands A and B is as follows:

```
LDA A     Load first operand A
CMA       Complement to get Ā
STA TMP   Store in a temporary location
LDA B     Load second operand B
CMA       Complement to get B̄
AND TMP   AND with Ā to get Ā ∧ B̄
CMA       Complement again to get A ∨ B
```

The other logic operations can be implemented by software in a similar fashion.

Subroutine

Frequently, the same piece of code must be written over again in many different parts of a program. Instead of repeating the code every time it is needed, there is an obvious advantage if the common instructions are written only once. **A set of common instructions that can be used in a program many times is called a subroutine.** Each time that a subroutine is used in the main part of the program, a branch is executed to the beginning of the subroutine. After the subroutine has been executed, a branch is made back to the main program.

Program: Write a subroutine that shifts the content of the accumulator four times to the left.

Location			
		ORG 100	/Main program
100		LDA X	/Load X
101		BSA SH4	/Branch to subroutine
102		STA X	/Store shifted number
103		LDA Y	/Load Y
104		BSA SH4	/Branch to subroutine again
105		STA Y	/Store shifted number
106		HLT	
107	X,	HEX 1234	
108	Y,	HEX 4321	
			/Subroutine to shift left 4 times
109	SH4,	HEX 0	/Store return address here
10A		CIL	/Circulate left once
10B		CIL	
10C		CIL	
10D		CIL	/Circulate left fourth time
10E		AND MSK	/Set *AC*(13–16) to zero
10F		BUN SH4 I	/Return to main program
110	MSK,	HEX FFF0	/Mask operand
		END	

In the basic computer, the link between the main program and a subroutine is the BSA instruction (Branch and Save return Address). To explain how this instruction is used, let us write a subroutine that shifts the content of the accumulator four times to the left.

The program of starts by loading the value of X into the AC. The next instruction encountered is BSA SH4. The BSA instruction is in location 101. Subroutine SH4 must return to location 102 after it finishes its task. When the BSA instruction is executed, the control unit stores the return address 102 into the location defined by the symbolic address SH4 (which is 109). It also transfers the value of SH4 + 1 into the program counter. After this instruction is executed, memory location 109 contains the binary equivalent of hexadecimal 102 and the program counter contains the binary equivalent of hexadecimal 10A. This action has saved the return address and the subroutine is now executed starting from location 10A (since this is the content of PC in the next fetch cycle).

The computation in the subroutine circulates the content of AC four times to the left. In order to accomplish a logical shift operation, the four low-order bits must be set to zero. This is done by masking

FFF0 with the content of AC. A mask operation is a logic AND operation that clears the bits of the AC where the mask operand is zero and leaves the bits of the AC unchanged where the mask operand bits are 1's.

The last instruction in the subroutine returns the computer to the main program. This is accomplished by the indirect branch instruction with an address symbol identical to the symbol used for the subroutine name. The address to which the computer branches is not SH4 but the value found in location SH4 because this is an indirect address instruction. What is found in location SH4 is the return address 102 which was previously stored there by the BSA instruction. The computer returns to execute the instruction in location 102. The main program continues by storing the shifted number into location X. A new number is then loaded into the AC from location Y, and another branch is made to the subroutine. This time location SH4 will contain the return address 105 since this is now the location of the next instruction after BSA. The new operand is shifted and the subroutine returns to the main program at location 105.

The BSA instruction performs an operation commonly called subroutine call. The last instruction of the subroutine performs an operation commonly called subroutine return.

Input-Output Programming

Users of the computer write programs with symbols that are defined by the programming language employed. The symbols are strings of characters and each character is assigned an 8-bit code so that it can be stored in computer memory. A binary-coded character enters the computer when an INP (input) instruction is executed. A binary-coded character is transferred to the output device when an OUT (output) instruction is executed. The output device detects the binary code and types the corresponding character.

Table (a) below lists the instructions needed to input a character and store it in memory. The SKI instruction checks the input flag to see if a character is available for transfer. The next instruction is skipped if the input flag bit is 1. The INP instruction transfers the binary-coded character into AC(0-7). The character is then printed by means of the OUT instruction. A terminal unit that communicates directly with a computer does not print the character when a key is depressed. To type it, it is necessary to send an OUT instruction for the printer. In this way, the user is ensured that the correct transfer has occurred. If the SKI instruction finds the flag bit at 0, the next instruction in sequence is executed. This instruction is a branch to return and check the flag bit again. Because the input device is much slower than the computer, the two instructions in the loop will be executed many times before a character is transferred into the accumulator.

Table (b) lists the instructions needed to print a character initially stored in memory. The character is first loaded into the AC. The output flag is then checked. If it is 0, the computer remains in a two-instruction loop checking the flag bit. When the flag changes to 1, the character is transferred from the accumulator to the printer.

TABLE: Programs to Input and Output One Character

```
(a) Input a character:
    CIF,    SKI          /Check input flag
            BUN CIF      /Flag=0, branch to check again
            INP          /Flag=1, input character
            OUT          /Print character
            STA CHR      /Store character
            HLT
    CHR,    —            /Store character here
(b) Output one character:
            LDA CHR      /Load character into AC
    COF,    SKO          /Check output flag
            BUN COF      /Flag=0, branch to check again
            OUT          /Flag=1, output character
            HLT
    CHR,    HEX 0057     /Character is "W"
```

Character Manipulation

A computer is not just a calculator but also a symbol manipulator. The binary coded characters that represent symbols can be manipulated by computer instructions to achieve various data-processing tasks. One such task may be to pack two characters in one word. This is convenient because each character occupies 8 bits and a memory word contains 16 bits. The program in Table lists a subroutine named IN2 that inputs two characters and packs them into one 16-bit word. The packed word remains in the accumulator. Note that subroutine SH4 (Table) is called twice to shift the accumulator left eight times.

Table: Subroutine to input and pack two characters in one word

```
IN2,    —            /Subroutine entry
FST,    SKI
        BUN FST
        INP          /Input first character
        OUT
        BSA SH4      /Shift left four times
        BSA SH4      /Shift left four more times
SCD,    SKI
        BUN SCD
        INP          /Input second character
        OUT
        BUN IN2 I    /Return
```

Subroutine

```
                        /Subroutine to shift left 4 times
SH4,    HEX 0           /Store return address here
        CIL             /Circulate left once
        CIL
        CIL
        CIL             /Circulate left fourth time
        AND MSK         /Set AC(13–16) to zero
        BUN SH4 I       /Return to main program
MSK,    HEX FFF0        /Mask operand
        END
```

Microprogrammed Control

Microprogramming is a process of writing microcode for a microprocessor. Microcode is low level code that defines how a microprocessor should function when it executes machine language instructions. Typically, one machine language instruction translates into several microcode instruction, on some computers, the microcode is stored in ROM and can not be modified.

Basic Concepts of Microprogramming:

Control word (CW): A word with each bit for one of the control signals. Each step of the instruction execution is represented by a control word with all of the bits corresponding to the control signals needed for the step set to one.

Microinstruction: Each step in a sequence of steps in the execution of a certain machine instruction is considered as a microinstruction, and it is represented by a control word. All of the bits corresponding to the control signals that need to be asserted in this step are set to 1, and all others are set to 0.

Microprogram: Composed of a sequence of microinstructions corresponding to the sequence of steps in the execution of a given machine instruction.

Microprogramming: The method of generating the control signals by properly setting the individual bits in a control word of a step.

Microoperations: In computer central processing units, micro-operations are detailed low level instructions used in some designs to implement complex machine instructions.

CONTROL MEMORY

In a digital computers, the function of the control unit is to initiate sequences of micro-operations. When the control signals are generated by hardware using conventional logic design techniques, the control unit is said to be hardwired. Microprogramming is a second alternative for designing the control unit of a digital computer.

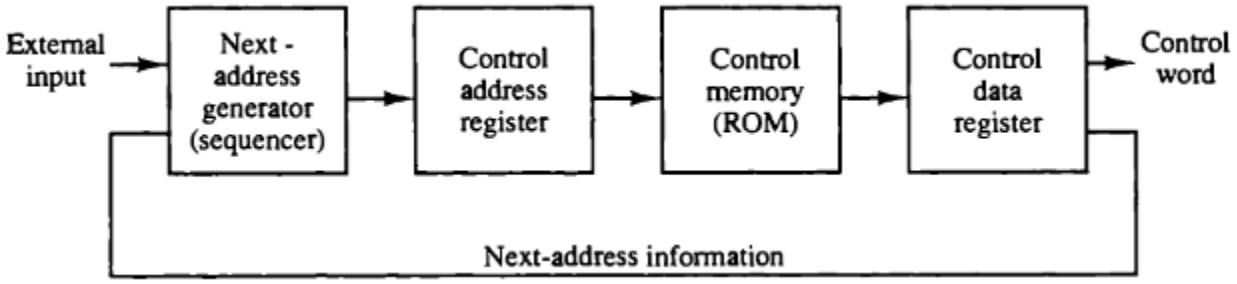
The control function that specifies a micro-operation is a binary variable. When it is in one binary state, the corresponding micro-operation is executed. A control variable in the opposite binary state does not change the state of the registers in the system. The active state of a control variable may be either the

1 state or the 0 state, depending on the application. In a bus organized system, the control signals that specify micro-operations are groups of bits that select the paths in multiplexers, decoders, and arithmetic logic units.

The control unit initiates a series of sequential steps of micro-operations. During any given time, certain micro-operations are to be initiated, while others remain idle. The control variables at any given time can be represented by a string of 1's and 0's called a control word. As such, control words can be programmed to perform various operations on the components of the system.

A control unit whose binary control variables are stored in memory is called a micro programmed control unit. Each word in control memory contains within it a microinstruction. The microinstruction specifies one or more micro-operations for the system. A sequence of microinstructions constitutes a microprogram. The control memory can be a read-only memory (ROM). The content of the words in ROM are fixed and cannot be altered by simple programming since no writing capability is available in the ROM.

The general configuration of a micro programmed control units is demonstrated in the block diagram. The control memory is assumed to be a ROM, within which all control information is permanently stored. The control memory address register specifies the address of the microinstruction, and the control data register holds the microinstruction read from memory.



Micro-programmed control organization

ADDRESS SEQUENCING

Microinstructions are stored in control memory in groups, with each group specifying routine. Each computer instruction has its own microprogram routine in control memory to generate the microoperations that execute the instruction.

The hardware that controls the address sequencing of the control memory must be capable of sequencing the microinstructions within a routine and be able to branch from one routine to another.

To appreciate the address sequencing in a micro-program control unit, let us specify the steps that the control must undergo during the execution of a single computer instruction.

Step-1:

- An initial address is loaded into the control address register when power is turned on in the computer.
- This address is usually the address of the first microinstruction that activates the instruction fetch routine.

- The fetch routine may be sequenced by incrementing the control address register through the rest of its microinstructions.
- At the end of the fetch routine, the instruction is in the instruction register of the computer.

Step-2:

- The control memory next must go through the routine that determines the effective address of the operand.
- A machine instruction may have bits that specify various addressing modes, such as indirect address and index registers.
- The effective address computation routine in control memory can be reached through a branch microinstruction, which is conditioned on the status of the mode bits of the instruction.
- When the effective address computation routine is completed, the address of the operand is available in the memory address register.

Step-3:

- The next step is to generate the microoperations that execute the instruction fetched from memory.
- The microoperation steps to be generated in processor registers depend on the operation code part of the instruction.
- Each instruction has its own micro-program routine stored in a given location of control memory.
- The transformation from the instruction code bits to an address in control memory where the routine is located is referred to as a mapping process.
- A mapping procedure is a rule that transforms the instruction code into a control memory address.

Step-4:

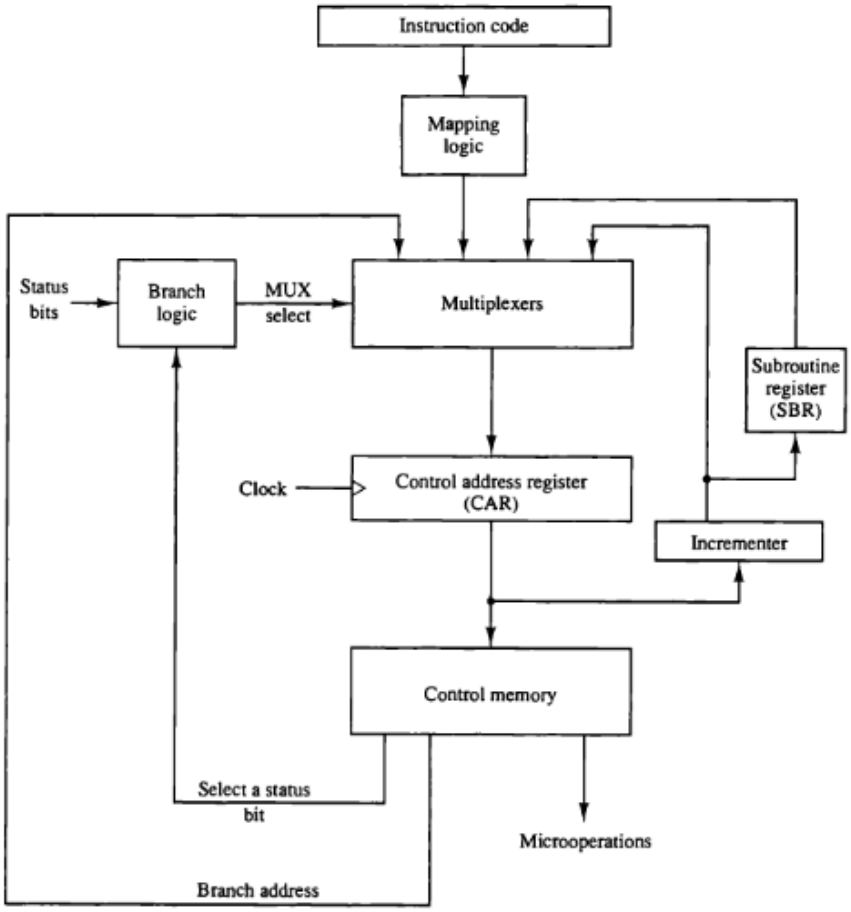
- Once the required routine is reached, the microinstructions that execute the instruction may be sequenced by incrementing the control address register.
- Micro-programs that employ subroutines will require an external register for storing the return address.
- Return addresses cannot be stored in ROM because the unit has no writing capability.
- When the execution of the instruction is completed, control must return to the fetch routine.
- This is accomplished by executing an unconditional branch microinstruction to the first address of the fetch routine.

In summary, the address sequencing capabilities required in a control memory are:

1. Incrementing of the control address register.
2. Unconditional branch or conditional branch, depending on status bit conditions.
3. A mapping process from the bits of the instruction to an address for control memory.
4. A facility for subroutine call and return.

Selection of Address for Control Memory

- Figure shows a block diagram of a control memory and the associated hardware needed for selecting the next microinstruction address.
- The microinstruction in control memory contains a set of bits to initiate microoperations in computer registers and other bits to specify the method by which the next address is obtained.
- The diagram shows four different paths from which the control address register (CAR) receives the address.



Selection of address of control memory

- The incrementer increments the content of the control address register by one, to select the next microinstruction in sequence.
- Branching is achieved by specifying the branch address in one of the fields of the microinstruction. Conditional branching is obtained by using part of the microinstruction to select a specific status bit in order to determine its condition.
- An external address is transferred into control memory via a mapping logic circuit.
- The return address for a subroutine is stored in a special register whose value is then used when the micro-program wishes to return from the subroutine.

Conditional Branching:

- The branch logic of above figure provides decision-making capabilities in the control unit.

- The status conditions are special bits in the system that provide parameter information such as the carry-out of an adder, the sign bit of a number, the mode bits of an instruction, and input or output status conditions.
- The status bits, together with the field in the microinstruction that specifies a branch address, control the conditional branch decisions generated in the branch logic.
- A 1 output in the multiplexer generates a control signal to transfer the branch address from the microinstruction into the control address register.
- A 0 output in the multiplexer causes the address register to be incremented.

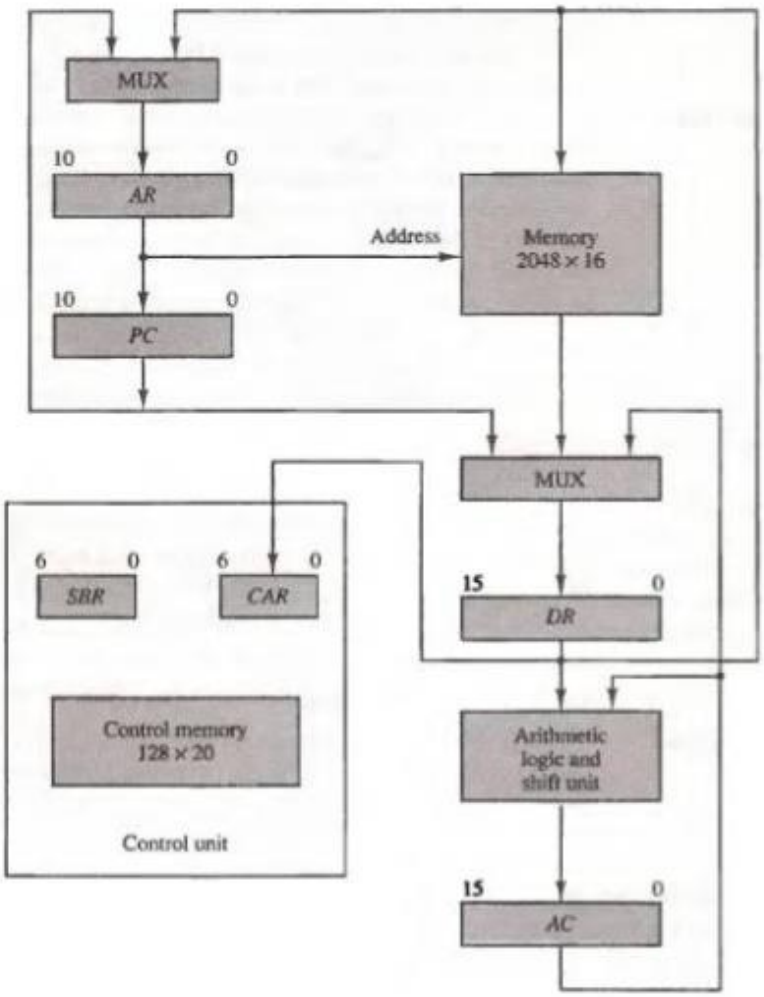
Microprogram Example

Once the configuration of a computer and its microprogrammed control unit is established, the designer's task is to generate the microcode for the control memory. This code generation is called microprogramming and is a process similar to conventional machine language programming.

Computer Configuration

The block diagram of the computer is shown in Fig. It consists of two memory units: a main memory for storing instructions and data, and a control memory for storing the microprogram. Four registers are associated with the processor unit and two with the control unit. The processor registers are program counter PC, address register AR, data register DR, and accumulator register AC. The control unit has a control address register CAR and a subroutine register SBR.

The transfer of information among the registers in the processor is done through multiplexers rather than a common bus. DR can receive information from AC, PC, or memory. AR can receive information from PC or DR. PC can receive information only from AR. The arithmetic, logic, and shift unit performs microoperations with data from AC and DR and places the result in AC. Note that memory receives its address from AR. Input data written to memory come from DR, and data read from memory can go only to DR.



Computer Hardware Configuration

Instruction format

The computer instruction format is depicted in Fig. It consists of three fields: a 1-bit field for indirect addressing symbolized by I, a 4-bit operation code (opcode), and an 11-bit address field. Figure (b) lists four of the 16 possible memory-reference instructions. The ADD instruction adds the content of the operand found in the effective address to the content of AC. The BRANCH instruction causes a branch to the effective address if the operand in AC is negative. The STORE instruction transfers the content of AC into the memory word specified by the effective address. The EXCHANGE instruction swaps the data between AC and the memory word specified by the effective address.

15	14 — 11	10 — 0
I	Opcode	Address

(a) Instruction format

Symbol	Opcode	Description
ADD	0000	$AC \leftarrow AC + M[EA]$
BRANCH	0001	If $(AC < 0)$ then $(PC \leftarrow EA)$
STORE	0010	$M[EA] \leftarrow AC$
EXCHANGE	0011	$AC \leftarrow M[EA], M[EA] \leftarrow AC$

EA is the effective address

(b) Four computer instructions

Instruction format

Microinstruction Format

The microinstruction format for the control memory is shown in Fig. The 20 bits of the microinstruction are divided into four functional parts. The three fields F1, F2, and F3 specify microoperations for the computer. The CD field selects status bit conditions. The BR field specifies the type of branch to be used. The AD field contains a branch address. The address field is seven bits wide, since the control memory has $128 = 2^7$ words.

3	3	3	2	2	7
F1	F2	F3	CD	BR	AD

F1, F2, F3: Microoperation fields

CD: Condition for branching

BR: Branch field

AD: Address field

Microinstruction code format (20 bits)

Symbols and Binary Code for Microinstruction Fields

F1	Microoperation	Symbol
000	None	NOP
001	$AC \leftarrow AC + DR$	ADD
010	$AC \leftarrow 0$	CLRAC
011	$AC \leftarrow AC + 1$	INCAC
100	$AC \leftarrow DR$	DRTAC
101	$AR \leftarrow DR(0\text{–}10)$	DRTAR
110	$AR \leftarrow PC$	PCTAR
111	$M[AR] \leftarrow DR$	WRITE

F2	Microoperation	Symbol
000	None	NOP
001	$AC \leftarrow AC - DR$	SUB
010	$AC \leftarrow AC \vee DR$	OR
011	$AC \leftarrow AC \wedge DR$	AND
100	$DR \leftarrow M[AR]$	READ
101	$DR \leftarrow AC$	ACTDR
110	$DR \leftarrow DR + 1$	INCDR
111	$DR(0\text{-}10) \leftarrow PC$	PCTDR

F3	Microoperation	Symbol
000	None	NOP
001	$AC \leftarrow AC \oplus DR$	XOR
010	$AC \leftarrow \overline{AC}$	COM
011	$AC \leftarrow \text{shl } AC$	SHL
100	$AC \leftarrow \text{shr } AC$	SHR
101	$PC \leftarrow PC + 1$	INCPC
110	$PC \leftarrow AR$	ARTPC
111	Reserved	

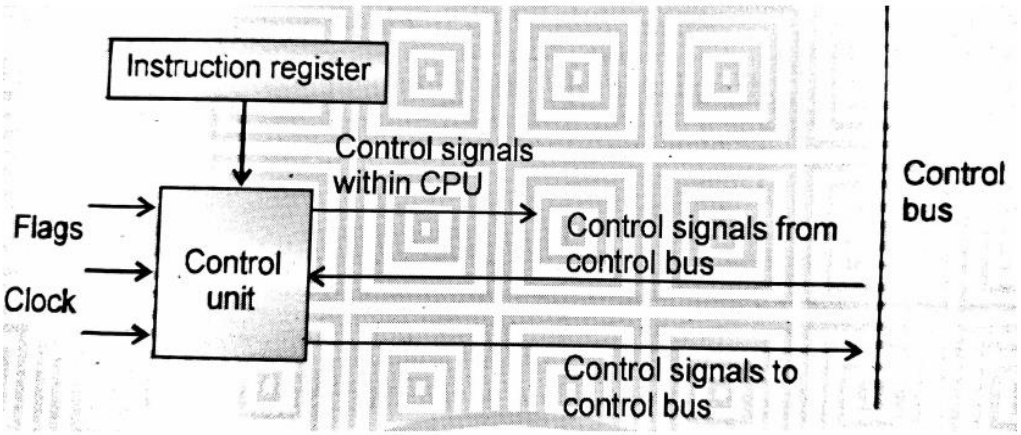
CD	Condition	Symbol	Comments
00	Always = 1	U	Unconditional branch
01	$DR(15)$	I	Indirect address bit
10	$AC(15)$	S	Sign bit of AC
11	$AC = 0$	Z	Zero value in AC

BR	Symbol	Function
00	JMP	$CAR \leftarrow AD$ if condition = 1
		$CAR \leftarrow CAR + 1$ if condition = 0
01	CALL	$CAR \leftarrow AD$, $SBR \leftarrow CAR + 1$ if condition = 1
		$CAR \leftarrow CAR + 1$ if condition = 0
10	RET	$CAR \leftarrow SBR$ (Return from subroutine)
11	MAP	$CAR(2\text{–}5) \leftarrow DR(11\text{–}14)$, $CAR(0,1,6) \leftarrow 0$

DESIGN OF CONTROL UNIT

Control unit generates timing and control signals for the operations of the computer. The control unit communicates with ALU and main memory. It also controls the transmission between processor, memory and the various peripherals. It also instructs the ALU which operation has to be performed on data.

A control unit works by receiving input information to which it converts into control signals, which are then sent to the central processor: The computer's processor then tells the attached hardware what operations to perform.



Block diagram of the control unit

(a) Clock: This is how the control unit 'keeps time.' The control unit causes one micro-operation to be performed for each clock pulse. This is referred as processor cycle time or clock cycle time.

(b) Instruction registers: The opcode of current instruction is used to determine which micro-operations to perform during the execute cycle.

(c) Flags: Used to determine the status of the processor and outcome of previous ALU operations.

(d) Control signals from control bus: The control bus portion of system bus provides signals to the control unit.

(e) Control signals within the processor:

1. Those that cause data to be moved from a register to another.
2. Those that activate specific ALU functions

(f) Control signals to control bus:

1. Control signals to memory
2. Control signals to input-output modules.

Totally, there are three types of control signals:

1. Those that activate ALU function.
2. Those that activate a data path.
3. Those that are signals on the external system bus.

Examples of devices that require a control unit are:

Control Processing Units (CPUs)
Graphics Processing Units (GPUs)

Functions of the Control Unit

1. It coordinates the sequence of data movements into, out of, and between a processor's many sub-units.
2. It interprets instructions.
3. It controls data flow inside the processor.
4. It receives external instructions or commands to which it converts to sequence of control signals.
5. It controls many execution units (i.e. ALU, data buffers and registers) contained within a CPU.
6. It also handles multiple tasks, such as fetching, decoding, execution handling and storing results.

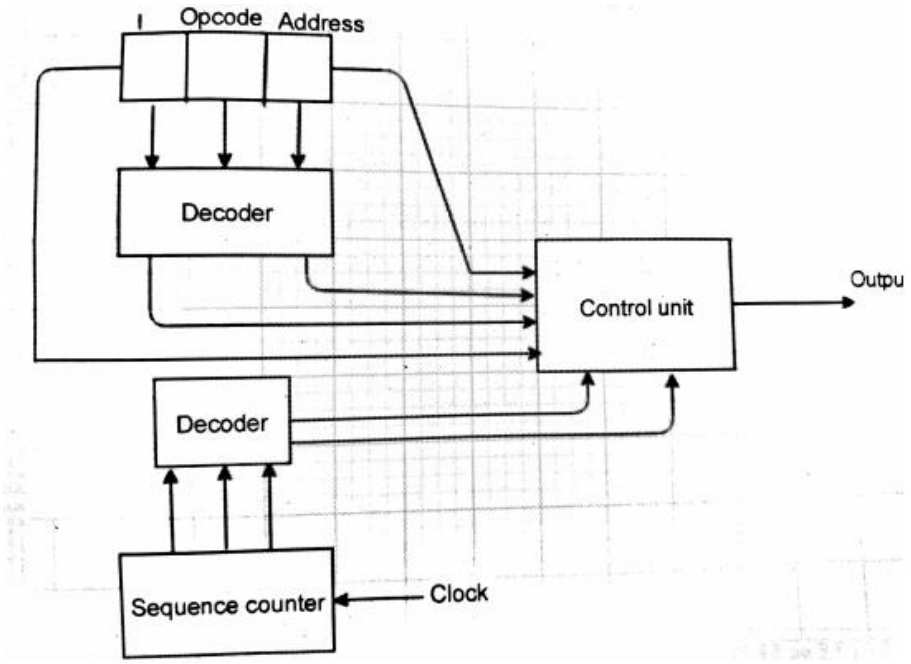
Type of control unit designing methods

1. Hardwired Control Unit

2. Microprogrammed Control Unit

Hardwired Control Unit

It is implemented with the help of gates, flip flops, decoders etc. in the hardware. The inputs to control unit are the instruction register, flags, timing signals etc. This organization can be very complicated if we have to make the control unit large. If the design has to be modified or changed, all the combinational circuits have to be modified which is a very difficult task.



Hard-wired Control Unit

- A Hard-wired Control consists of two decoders, a sequence counter, and a number of logic gates.
- An instruction fetched from the memory unit is placed in the instruction register (IR).
- The component of an instruction register includes; I bit, the operation code, and bits 0 through 11.
- The operation code in bits 12 through 14 are coded with a 3 x 8 decoder.
- The outputs of the decoder are designated by the symbols D0 through D7.

- The operation code at bit 15 is transferred to a flip-flop designated by the symbol I.
- The operation codes from Bits 0 through 11 are applied to the control logic gates.
- The Sequence counter (SC) can count in binary from 0 through 15.

Advantages of Hardwired Control unit:

1. It is faster than the microprogrammed control unit.
2. It can be optimized to produce the fast mode of operation.

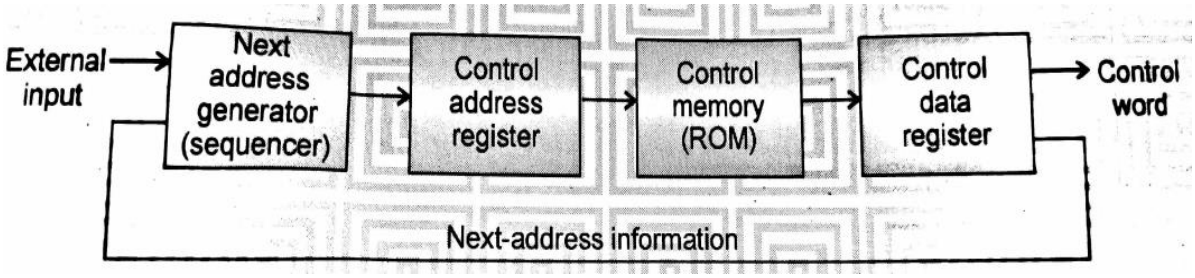
Disadvantages of Hardwired control unit:

1. Instruction set, the control logic is directly implemented.
2. Requires change in wiring if the design has to be controlled.
3. An occurrence of an error is more.
4. Complex decoding and sequencing logic.
5. It requires a more chip area, therefore, it is a costlier control unit.

Microprogrammed Control Unit

It is implemented by using programming approach. A sequence of micro operations is carried out by executing a program consisting of micro-instructions. In this organization any modifications or changes can be done by updating the micro program in the control memory by the programmer.

In microprogrammed control units, subsequent instruction words are fetched into the instruction register in a normal way. However, the operation code of each instruction is not directly decoded to enable immediate control signal generation but it comprises the initial address of a microprogram contained in the control store.



Microprogrammed control units

Microprogrammed control units a basic computer:

- The Control memory address register specifies the address of the micro-instruction.
- The Control memory is assumed to be a ROM, within which all control information is permanently stored.
- The control register holds the microinstruction fetched from the memory.
- The micro-instruction contains a control word that specifies one or more micro-operations for the data processor.
- While the micro-operations are being executed, the next address is computed in the next address generator circuit and then transferred into the control address register to read the next microinstruction.

- The next address generator is often referred to as a micro-program sequencer, as it determines the address sequence that is read from control memory.

Advantages of Micro Programmed Control unit:

1. It is both cheaper and the occurrence of an error is less.
2. More flexible to accommodate with new instructions.
3. Easier decoding and sequencing can be done.
4. Easier to handle complex instruction sets.
5. It requires a less chip area.

Disadvantages of Micro programmed control unit:

This is slower than the hardwired control unit because the microinstructions are to be fetched from the control memory which is time-consuming.